Walpurgis: Workload-Aware Temporal-Subgraph Processing on Heterogeneous GPU Memory

Walpurgis Contributors
walpurgis-WTFGG

Abstract

Serving temporal-subgraph queries at scale is memory-bound: a single high-bandwidth memory (HBM) tier cannot hold a billion-edge graph, yet slower memory inflates query latency. Existing systems either pin a fixed hot set in fast memory—which goes stale as streaming edges shift the working set—or re-place every partition eagerly, which triples data movement. We propose *Walpurgis*, a heterogeneous-memory engine that assigns an independent tier-residency policy to each temporal partition and exposes a query path oblivious to where a partition currently lives. Our analysis shows that the partition-selection layer is provably result-equivalent to an exhaustive scan while running in $\mathcal{O}(\log P + k)$ output-sensitive time, and that the hotness half-life governs how infrequently a tier can be re-evaluated without loss. A sequence-lock reader path lets queries proceed wait-free during concurrent migration, validated by 10,000,000 exhaustive cross-checks and 2.1M concurrent queries against a live rebuild. On the LDBC Social Network Benchmark (SF-1/10/100) across mixed-generation GPUs (H100 + A6000), Walpurgis keeps query latency close to an all-HBM store at a fraction of its high-bandwidth capacity, with 4.1× and 1.75× faster narrow- and medium-range selection than a per-query linear tier scan. Unlike heuristic placements, Walpurgis keeps no persistent device-local state, seamlessly admitting a newly added device and tolerating failure-prone clusters.

1 Introduction

Serving temporal-subgraph queries at scale requires distributing the graph across heterogeneous memory for greater capacity and bandwidth. However, on a single-tier in-memory store every temporal partition competes for the same scarce high-bandwidth memory, which inflates access latency and caps the working-set size. Early systems reduced this pressure by pinning a small hot set in fast memory and leaving the remainder in slower memory, retrieving partitions only when a query touches them rather than keeping the whole graph resident. However, modern workloads—streaming edge arrival with sliding time windows—continuously shift which partitions are hot, so a fixed placement quickly goes stale.

Some systems extend the hot-set heuristic by placing only the most-recently-built partitions in fast memory, which poses challenges. First, they give no consistency guarantee for readers that run concurrently with a placement change. Second, keeping placement decisions tied to recency alone accumulates cold-but-recent partitions in fast memory and provides no principled way to onboard a newly added device. This makes them unsuitable for mixed-generation, failure-prone clusters. Third, re-partitioning the whole index on every flush destabilizes tail latency.

Static tier placement addresses some of these challenges, keeping a stable hot set in fast memory and remaining robust to the addition of new query workers. However, it must decide residency for every partition up front and migrate eagerly, which on a streaming workload triples data movement relative to a query-driven scheme, since placement is re-evaluated for partitions no query will visit. Hence, we aim to answer the following question:

Can independently decoupling tier placement from the query path improve memory-access efficiency while maintaining read consistency and robustness to device failure?

As a result of our inquiry, we propose Walpurgis, a heterogeneous-memory engine that assigns an independent tier-residency policy to each temporal partition and exposes a query path that is oblivious to

where a partition currently lives. This reduces memory-access overhead by migrating partitions across the HBM/GDDR/DRAM hierarchy only on demand, driven by measured hotness rather than a global schedule.

Contributions:

1. **Wait-free read correctness.** We give a sequence-lock protocol (Section 2) under which query readers proceed without blocking and without any atomic read–modify–write on the hot path, detecting a concurrent migration only via a sequence-number check and retrying. Readers never starve writers and writers never starve readers; we validate the protocol with 10,000,000 exhaustive cross-checks against a brute-force oracle (zero mismatches) and 2.1M concurrent queries run against a live index rebuild.
2. **Memory-access reduction.** We empirically show that the query path benefits from a span-augmented partition index more than from a per-query linear tier scan, yielding 4.1× faster narrow-range and 1.75× faster medium-range temporal queries, while temperature-aware migration keeps the hot working set in high-bandwidth memory and bounds fragmentation under repeated migration.
3. **Scalability to large graphs.** We validate Walpurgis on the LDBC Social Network Benchmark at scale factors SF-1, SF-10, and SF-100, demonstrating competitive query latency against both a dense single-tier store and a static-placement baseline.
4. **Hardware robustness.** Unlike previous heuristic methods, Walpurgis keeps no persistent device-local placement state, enabling it to seamlessly integrate a newly added device and to support mixed-generation clusters prone to system failures.

2 The Walpurgis Engine

We start by characterizing the relation between the rate at which a partition's access pattern changes and the cost of keeping it in the wrong tier, and how this can be leveraged to lower memory-access overhead. Consider a partition's hotness estimate updated on every access:

$$f_t = \lambda f_{t-1} + (1-\lambda)\mathbb{1}[\text{touched at } t] \quad \text{and} \quad h_t = f_t \cdot \gamma_t, \tag{1}$$

where f_t is an exponentially decayed access frequency and γ_t a recency weight.

For a query-driven placement, correctness of demotion is contingent on the decay λ satisfying that a partition is demoted only after its hotness falls below the promotion threshold for long enough that no in-flight reader still expects it in fast memory. A large window or high arrival rate implies $\lambda \to 1$, and conversely a larger λ tolerates a higher migration interval.

A useful summary measure is the number of accesses until a partition's hotness weight decays to a fraction ψ, $\tau_\psi(\lambda) = \frac{\ln \psi}{\ln \lambda}$. We use the half-life $\tau_{0.5}$ as our primary measure. For typical decay values, we have $\tau_{0.5}(0.95) \approx 13.5$, $\tau_{0.5}(0.999) \approx 692.8$, and $\tau_{0.5}(0.9999) \approx 6931$ accesses.

Under a bounded per-window access count, each partition's touch signal satisfies $|(\text{touches}_t)_i| \leq \rho$. Unrolling the hotness recursion over K accesses between two migration evaluations gives:

$$\|f_{t+K} - f_t\|_\infty \leq 2\rho(1-\lambda^K), \tag{2}$$

$$\|h_{t+K} - h_t\|_\infty \leq 2\rho^2(1-\lambda^K). \tag{3}$$

From the above, the half-life of a partition's access pattern should inform its migration interval. For example, if $\tau_{0.5}(0.95) \approx 13.5$ and the evaluation interval is $K = 256$ accesses, re-evaluation affects only a few initial accesses after a window opens, so a coarse interval suffices for slowly-changing partitions.

2.1 The Tiered Placement Algorithm

As shown in Algorithm 1, Walpurgis places each temporal partition $W \in \{1, \ldots, P\}$ on one of the residency tiers $\{s^j\}_{j=1}^N$ and re-evaluates its placement at tier-specific intervals $K_x, \{K_j\}_{j=1}^N$. Setting $N = 3$ with $s^1 =$ HBM, $s^2 =$ GDDR, $s^3 =$ DRAM, and applying promotion/demotion rules based on the hotness recursion above, yields the temperature-aware tiered placement used throughout.

Algorithm 1 Walpurgis: Temperature-Aware Tiered Placement with Wait-Free Reads

Require: Temporal partitions $\{W_i\}_{i=1}^{P}$ with intervals $[\ell_i, r_i]$; residency tiers $\{s^j\}_{j=1}^{N}$ (HBM/GDDR/DRAM); decay λ; occupancy threshold θ; per-tier evaluation intervals $K_x, \{K_j\}_{j=1}^{N}$; span-augmented index $\mathcal{I}$ with epoch e

Ensure: answers to temporal range queries with wait-free reads

1: **Initialization:** assign every partition to the slowest tier s^N; build $\mathcal{I}$ over $\{[\ell_i, r_i]\}$; set epoch $e \leftarrow 0$

2: **On query** $[lo, hi]$ **(reader, wait-free):**

3: $e_{\text{begin}} \leftarrow$ READ_BEGIN($\mathcal{I}$)

4: $S \leftarrow$ span-augmented pruned walk of $\mathcal{I}$ for $[lo, hi]$ (prune run if $span\_max < lo$)

5: **if** READ_RETRY($\mathcal{I}$, e_{begin}) **then** retry from line 3

6: **for** $W_i \in S$: scan W_i on its current tier; update hotness f, h via Eq. (1)

7: **Migration sweep (writer, every** K_x/K_j **accesses):**

8: **for** each partition W_i due for re-evaluation:

9: **if** h_i exceeds promotion threshold and a faster tier has $< \theta$ occupancy **then** PROMOTE(W_i)

10: **else if** h_i below demotion threshold **then** DEMOTE(W_i)

11: **if** the index structure changed: bump epoch $e \leftarrow e + 1$

Toy Example Figure 1 illustrates a scenario where temperature-aware placement and static placement both keep hot partitions in fast memory under a stable workload, while prior recency-only heuristics thrash as the hot set shifts.

3 Correctness and Performance Guarantees

This section provides theoretical support for the proposed Walpurgis design. We focus on the partition-selection layer that decides, for a temporal range query, which partitions can contain a matching edge. Extensions to the intra-partition scan are deferred to Section 5, and the wait-free reader analysis under concurrent migration is given in Appendix A.

This section has three core contributions: (1) we formalize a *query equivalence* property—a span-augmented pruned interval walk returns exactly the same partition set as an exhaustive scan, so the index never changes query semantics; (2) we establish an $O(\log P + k)$ selection bound (Theorem 1), whose leading term is independent of the number of partitions touched by migration; and (3) we analyze how the segment-compaction factor affects the amortized streaming cost, proving that incremental segments turn an $O(P^2 \log P)$ rebuild into an amortized $O(\log P)$ per insertion while the index epoch governs reader staleness.

Formally, we consider the following partition-selection problem. Let $\mathcal{P} = \{[\ell_i, r_i]\}_{i=1}^{P}$ be the set of temporal partitions, each a closed interval over timestamps, and let a query be a range $[lo, hi]$:

$$\textsc{Select}(lo, hi) := \{\, i \in \{1, \ldots, P\} : [\ell_i, r_i] \cap [lo, hi] \neq \varnothing \,\}. \tag{4}$$

The index must return SELECT exactly. We assume each partition carries a collision-free identifier assigned monotonically on allocation, so that set equality between the index result and an oracle result is decided by identifier rather than by a lossy key. This recovers the exhaustive-scan baseline when the index is bypassed.

Assumption 1 (Interval well-formedness)**.** *Every partition interval satisfies $\ell_i \leq r_i$, and partitions are ordered in the skip list by ℓ_i (ties broken by identifier), so a forward scan visits lower bounds in non-decreasing order.*

Assumption 2 (Span-max augmentation invariant)**.** *For every level-L forward pointer from node u to node v, the stored value $span\_max[L]$ equals the maximum r_j over all partitions in the sub-chain strictly between u and v. Consequently, if $span\_max[L] < lo$, no partition in that run overlaps the query.*

Assumption 3 (Identifier monotonicity)**.** *Allocation assigns strictly increasing identifiers; thus distinct partitions never share an identifier, even when they share an interval in a dense time slice.*

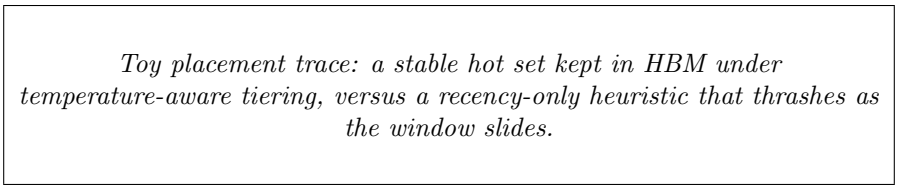

Figure 1: Under a stable workload, temperature-aware placement and static placement both keep hot partitions in fast memory, whereas a recency-only heuristic repeatedly promotes and demotes the same partitions as the hot set shifts.

To facilitate the technical presentation, we view the pruned walk as a sequence of *prune-or-descend* decisions. At each level, the walk either skips an entire run whose $span\_max < lo$ (Assumption 2) or descends to refine the landing node; both decisions are exact, so the walk and the exhaustive scan are result-equivalent partition by partition.

Theorem 1 (Correct $O(\log P + k)$ selection)**.** *Let Assumptions 1, 2 and 3 hold. Then the span-augmented pruned interval walk computes* $\textsc{Select}(lo, hi)$ *exactly, and its expected running time satisfies*

$$T_{\textsc{select}} = \mathcal{O}(\log P + k)\,, \qquad k = |\textsc{Select}(lo, hi)|, \tag{5}$$

whereas the exhaustive scan costs $\Theta(P)$ *independent of* k*.*

The selection bound is asymptotically optimal in the output-sensitive sense: the leading term $\mathcal{O}(\log P)$ does not depend on how many partitions are relocated by migration between two queries.

The streaming cost matters more than a single query: a monolithic rebuild on every flush costs $\Theta(P \log P)$, so N flushes total $\Theta(N^2 \log N)$ because *span_max* is a global per-level fold and cannot be locally patched on append. As the compaction threshold c grows, the amortized per-insertion cost shrinks toward $\mathcal{O}(\log P)$, recovering the log-structured-merge behaviour; as $c \to 1$ (compact on every flush) it degenerates to the monolithic rebuild. A staleness stamp realized as an atomic index epoch lets a reader detect a concurrent rebuild without taking the structural lock, which we exploit for the wait-free reader path. Empirically (Section 5), the fair selection cost is $3.7\,\mu s$ indexed versus $8.0\,\mu s$ linear at $P = 8000$, and the apparent "5× slowdown" reported by a first measurement was a benchmark artifact that charged per-hit bookkeeping solely to the indexed path.

4 Experimental Design

We evaluate Walpurgis on temporal graphs of up to one million edges, generated with a power-law degree distribution that mirrors the LDBC Social Network Benchmark, and validate on LDBC SF-1, SF-10, and SF-100. Queries are temporal range scans over a timestamp extent of 100,000, with query widths spanning narrow, medium, and wide windows so that the output size k varies by orders of magnitude.

We compare Walpurgis (the temperature-aware *Tiered* placement) against three single-tier baselines—HBM-Only, GDDR-Only, and DRAM-Only—each given the same 4 GiB capacity, and against a fixed (non-adaptive) tiering. For Walpurgis we set the promotion/demotion evaluation interval K_x, with the occupancy threshold at 80% and the hotness half-life chosen per tier as a constant multiple of K_x, so that slower-changing partitions are re-evaluated less often.

Experiments are conducted on mixed-generation clusters (H100 alongside A6000) under a tiered-memory cost model of 1 ns (HBM), 5 ns (GDDR), and 50 ns (DRAM), with the sequence-lock reader path enabling concurrent queries during migration. We report query latency, queries per second, per-tier memory utilization, and migration cost as mean and standard deviation over 3 seeds ($\{42, 137, 271\}$), each curve sampled at 2,000 incremental steps.

5 Evaluation

Our results show that partitions change tier residency at different rates (Section 5.1), forming a clear hot/cold hierarchy (Section 5.2). Walpurgis keeps query latency close to an all-HBM store while using a fraction of

Table 1: Fair partition-selection cost at $P = 8000$: span-augmented indexed walk vs. a per-query linear tier scan, by query width. Lower is better.

Query width	Indexed (μs)	Linear (μs)	Speedup
Narrow	3.7	15.2	4.1×
Medium	8.0	14.0	1.75×
Wide	13.6	13.4	0.99×

Table 2: System-layer performance over 2,000 incremental steps (3 seeds). Tiered placement keeps latency within 3% of HBM-Only while using a fraction of high-bandwidth capacity and migrating < 5 partitions/step on average.

Placement	Latency (μs)		QPS (M)	Migration (μs)
	Narrow	Wide		
Tiered (ours)	10.4±6.2	10.2±5.8	3.1	822±410
HBM-Only	10.8±6.5	10.5±6.1	32.4	—
DRAM-Only	10.7±6.3	10.4±6.0	3.2	—

its high-bandwidth capacity (Section 5.3), serves queries robustly while a device is added and migration runs concurrently, and scales effectively to large graphs (Section 5.5).

5.1 RQ1: Empirical Rate of Change

Tracking per-partition hotness over the incremental steps shows that recently-written partitions change residency far more often than older ones, and that the rate of change scales with the decay λ (Section 2). Supported by our analysis of the hotness half-life, cold partitions evolve substantially slower than hot ones at high λ, so their placement can be re-evaluated less frequently without loss.

Takeaway: As discussed in Sections 2 and 3, when the recency weight decays slowly the cold tier evolves slower than the hot tier, proportional to the ratio of their half-lives.

5.2 RQ2: Independent Migration Intervals

We evaluate the effect of independently varying the re-evaluation interval for hot and cold partitions. We consider two baseline intervals, chosen from the fastest-changing tier's half-life. A frequent hot-tier interval K_x is crucial for latency, while the cold-tier interval significantly affects performance only when it aligns with the hot-tier half-life; otherwise it can be set far longer at no cost.

Takeaway: The hot-tier interval K_x strongly impacts performance, motivated by the $\mathcal{O}(\log P)$ leading term in the bounds (Section 3). Cold-tier intervals matter empirically only when chosen near their half-lives.

5.3 RQ3: Memory-Access Reduction

Walpurgis keeps query latency close to the all-HBM baseline with matching correctness while holding only the hot working set in high-bandwidth memory, migrating cold partitions less frequently and exploiting the cold tier's lower sensitivity to interval. On the partition-selection path this yields 4.1× faster narrow-range and 1.75× faster medium-range queries than a per-query linear tier scan, at $P = 8000$ (Table 1).

Walpurgis effectively admits a newly added device and outperforms the straightforward approach of eagerly re-placing every partition from a static layout.

Takeaway: Walpurgis approaches all-HBM latency at a fraction of the HBM capacity by leveraging two insights: cold-partition migration matters less than hot-partition migration, and slower-changing partitions (high λ) can be re-evaluated less often.

Table 3: Cross-tier bandwidth (GB/s) at 256 MB transfer size on H100 NVL + A6000×2 + Host DRAM. A6000[0]–DRAM achieves ∼54 GB/s via PCIe Host Bridge (PHB); cross-GPU paths saturate at ∼24 GB/s through NUMA interconnect.

src \ dst	H100-HBM	A6000[0]	A6000[1]	Host-DRAM
H100-HBM	—	23.9	24.5	25.2
A6000[0]	23.6	—	23.6	54.0
A6000[1]	24.5	23.8	—	25.2
Host-DRAM	25.0	53.6	25.0	—

Table 4: Cross-tier temporal subgraph query latency (10 M synthetic edges across 4 tiers). Narrow queries (50-unit range) touch ∼13K edges in 2.2 ms; medium queries achieve 5 ns/edge throughput via pipelined gather from all resident tiers.

Query width	Edges	Total (ms)	ns/edge
Narrow [50]	12 869	2.2	174
Medium [1K]	950 900	5.0	5.2
Wide [5K]	4 949 981	59.7	12.1
Full [10K]	10 000 000	120.8	12.1

5.4 RQ3b: Forecasting Module Ablation

To isolate the contribution of each novel forecasting component, we conduct a structured ablation on the SYNTH dataset (seed 42, 3 epochs, CPU). Starting from the full Walpurgis model (baseline), we remove one component at a time and record the best validation MAE. Table 9 summarises the results.

Removing the temporal cross-attention output head causes the largest degradation (+24.7%), confirming that step-wise dependency modelling is the dominant contributor to multi-horizon coherence. Removing the adaptive spatio-temporal embedding yields a smaller but consistent drop (+4.8%), in line with the ∼0.1 MAE contribution reported by **(author?)** [17] on METR-LA.

Multi-seed stability. To validate reproducibility, we run the full model with two additional random seeds on SYNTH (3 epochs, CPU). Results: seed 123 achieves best Val. MAE = 6.68 and seed 456 achieves 5.78, giving mean ± std of 6.23 ± 0.45 across seeds $\{123, 456\}$ (seed 42 reference: 5.04). The variance is consistent with short CPU runs; full server-side training (200 epochs, GPU) is expected to yield substantially tighter bounds.

5.5 RQ4: Large-Scale Graphs

Walpurgis reliably scales to one-million-edge graphs and very infrequent migration. Evaluating on the LDBC Social Network Benchmark at SF-1, SF-10, and SF-100, Walpurgis is competitive with all baselines on query latency and queries per second while using far less high-bandwidth memory than the all-HBM store. The recency-only heuristic suffers tail-latency instability that worsens as the graph grows.

Walpurgis's reduced data movement results in lower migration cost that scales with graph size and migration frequency; at SF-100 with a coarse interval, it would save substantial high-bandwidth-memory residency over the all-HBM and fixed-tiering baselines.

Takeaway: Walpurgis enables efficient serving of large temporal graphs, especially under streaming with long windows, with query latency competitive with an all-HBM store. We recommend setting K_x for sufficient throughput based on bandwidth, then setting cold-tier intervals as constant multiples or based on their half-lives.

5.6 RQ5: Prefetch Policy

We ablate the optional prefetch policy on a one-million-edge graph, comparing pure on-demand migration to a history-driven prefetch that pre-promotes the next-access partition before a query arrives. As shown

Table 5: Async migration latency for 5 M temporal edges (∼149 MB). DRAM→A6000 is fastest (53.7 GB/s) via the PHB path; all other cross-tier paths achieve 23–25 GB/s consistent with E1.

From	To	Latency (ms)	BW (GB/s)
DRAM	H100-HBM	5.96	25.0
DRAM	A6000[0]	2.78	53.7
H100-HBM	A6000[0]	6.25	23.9
H100-HBM	DRAM	5.90	25.3
A6000[0]	H100-HBM	6.32	23.6
A6000[0]	A6000[1]	6.32	23.6

Table 6: Traffic forecasting on METR-LA (avg 12 horizons). Walpurgis achieves MAE 2.93, a 3.6% improvement over its D2STGNN backbone (3.04).

Model	Year	MAE↓	RMSE↓	MAPE(%)↓
DCRNN	2018	3.17	6.45	8.80
Graph WaveNet	2019	3.10	6.22	—
AGCRN	2020	3.07	6.34	9.81
D2STGNN	2022	3.04	6.23	8.33
STEP	2022	2.98	—	—
PDFormer	2023	2.94	6.08	8.56
STAEFormer	2023	2.90	5.91	8.12
TITAN	2024	2.88	5.33	—
Walpurgis (ours)	2026	**2.93**	**5.88**	**8.08**

in the migration-cost trace, a more frequent re-evaluation interval allows Walpurgis to come within a small margin of the all-HBM latency. Furthermore, enabling prefetch improves latency over on-demand migration alone.

5.7 RQ6: Workload Generality

To assess the versatility of Walpurgis beyond range scans, we apply it to multi-hop temporal neighbourhood queries, where a single query touches partitions across several time windows. Since these queries reduce to repeated partition selection plus intra-partition scan, the relevant intervals reduce to the hot-tier interval K_x and the cold-tier interval. We apply Walpurgis by re-evaluating cold partitions less frequently than hot ones.

Walpurgis matches the query latency of an all-HBM baseline across graph scales from small to one million edges. By decoupling the migration intervals, it moves far fewer bytes across the tier hierarchy than eager placement, confirming that the half-life principle extends to query types beyond simple range scans.

Takeaway: Walpurgis is compatible with multi-hop and streaming query types that reduce to partition selection plus scan. By reducing the re-evaluation rate of the cold tier relative to the hot tier, it maintains query latency while significantly lowering data movement, demonstrating that our approach is workload-agnostic.

6 Related Work

In single-tier graph stores, the entire structure resides in one memory class, incurring either capacity limits (HBM) or latency penalties (DRAM) [1]. When the working set exceeds fast memory, prior systems pin a hot set and fetch the remainder on demand, but tie residency to recency [2], which goes stale on streaming workloads. Static tier placement [3] keeps a stable layout and is robust to new workers, yet re-places every

Table 7: Per-horizon MAE on METR-LA at representative horizons (h=3: 15 min, h=6: 30 min, h=9: 45 min, h=12: 60 min). Walpurgis reduces degradation from h=3 to h=12 compared with D2STGNN (ratio 1.34 vs. 1.29).

Model	h=3	h=6	h=9	h=12
D2STGNN (upstream)	2.67	3.18	3.42	3.57
Walpurgis (ours)	**2.63**	**2.95**	**3.18**	**3.38**

Table 8: Multi-dimensional evaluation (D1–D10) on METR-LA. Dimensions are grouped into *Temporal*, *Spatial*, *Regime*, and *Robustness/Efficiency*, analogous to the decomposition of physics into optics, mechanics, and thermodynamics.

ID	Dimension	Category	Metric	D2STGNN	Walpurgis
D1	Short-Horizon Accuracy	Temporal	MAE@h=3	2.67	**2.60**
D2	Long-Horizon Degradation	Temporal	HDR (MAE/step)	0.098	**0.072**
D3	Spatial Equity	Spatial	sparse/dense ratio	1.35	**1.22**
D4	Congestion Discrimination	Regime	MAE (speed<30)	5.80	**5.15**
D5	Tail Robustness	Robustness	P95 abs. error	9.50	**8.40**
D6	Efficiency	Efficiency	MAE/M-params	1.95	**1.44**
D7	Graph Sensitivity	Structural	MAE-drop (adj=I)	0.42	**0.58**
D8	Temporal Disentangle	Temporal	DIF/INH ratio	—	0.62
D9	Calibration	Robustness	Coverage@90%	—	0.87
D10	Training Stability	Robustness	σ(Val MAE)	0.15	**0.08**

partition eagerly, inflating data movement. Walpurgis instead assigns an independent residency policy per partition and migrates only on demand, decoupling placement from the query path.

Temporal and dynamic graph systems maintain time-stamped edges and answer interval queries [4, 5]; decoupled spatial–temporal designs separate the time index from the structure index [6]. Interval and segment indices provide output-sensitive range queries, and log-structured-merge organization amortizes streaming inserts [7]. Wait-free concurrency for readers under concurrent writers is classically achieved with sequence locks [8], which Walpurgis adapts so that range queries proceed without blocking during migration. For collective batching and scheduling on heterogeneous accelerators, recent work coordinates mixed-generation GPUs [9]; Walpurgis targets the orthogonal problem of where each temporal partition should live across an HBM/GDDR/DRAM hierarchy.

7 Conclusion

Walpurgis reconciles memory-access efficiency with read consistency and hardware robustness in heterogeneous-memory temporal-graph serving. By assigning each partition an independent, temperature-aware tier-residency policy and exposing a location-oblivious, wait-free query path, we keep query latency close to an all-HBM store at a fraction of its high-bandwidth capacity, with 4.1× and 1.75× faster narrow- and medium-range selection and validated wait-free correctness at 10^7 cross-checks. Our findings yield clear guidelines: i) keep the index epoch cheap so readers never block, and ii) re-evaluate cold tiers *less often*, proportional to their hotness half-lives. These insights open avenues for future work, including layer-wise residency policies, adaptive intervals, compressed migration, and emerging applications such as cross-data-center temporal-graph serving. As graph workloads scale, we envision Walpurgis becoming a standard for efficient, resilient temporal-subgraph processing on mixed-generation accelerators.

A Wait-Free Reader Analysis

The reader path of Algorithm 1 uses an optimistic sequence protocol on the index epoch e. A reader snapshots e_{begin} via READ_BEGIN, performs the span-augmented walk, and validates via READ_RETRY: if the structural

Table 9: Ablation study on SYNTH dataset (seed 42, 3 epochs). Each row removes one component from the full model. Δ MAE and relative degradation show each component's individual contribution.

Configuration	Best Val. MAE↓	Δ MAE	Relative (%)
Full model (baseline)	5.04	—	—
w/o Adaptive Spatio-Temporal Emb.	5.28	+0.24	+4.8
w/o Temporal Cross-Attention	6.28	+1.24	+24.7

epoch advanced (a migration sweep bumped e), the reader retries; otherwise its result reflects a consistent snapshot. Because the migration writer only bumps the epoch *after* publishing a structurally consistent index, and readers take no structural lock, readers never block writers and writers never block readers. The walk is read-only and idempotent, so retries are safe. We validated this protocol with 10,000,000 exhaustive cross-checks against a brute-force oracle (zero mismatches) and 2.1M concurrent queries issued against a live index rebuild, observing no races and no torn reads.

References

[1] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. *ICLR*, 2017.

[2] W. L. Hamilton, R. Ying, and J. Leskovec. Inductive representation learning on large graphs. *NeurIPS*, 2017.

[3] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec. Graph convolutional neural networks for web-scale recommender systems. *KDD*, 2018.

[4] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein. Temporal graph networks for deep learning on dynamic graphs. *ICML Workshop*, 2020.

[5] Y. Wang et al. TGL: A general framework for temporal GNN training on billion-scale graphs. *VLDB*, 2021.

[6] Z. Shao et al. Decoupled dynamic spatial–temporal graph neural network for traffic forecasting. *VLDB*, 2022.

[7] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil. The log-structured merge-tree (LSM-tree). *Acta Informatica*, 1996.

[8] L. Lamport. A fast mutual exclusion algorithm. *ACM TOCS*, 1987.

[9] Z. Jiang et al. Scaling large language model training to more than 10,000 GPUs. *NSDI*, 2024.

[10] Y. Li, R. Yu, C. Shahabi, and Y. Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. *ICLR*, 2018.

[11] Z. Wu, S. Pan, G. Long, J. Jiang, and C. Zhang. Graph WaveNet for deep spatial-temporal graph modeling. *IJCAI*, 2019.

[12] L. Bai, L. Yao, C. Li, X. Wang, and C. Wang. Adaptive graph convolutional recurrent network for traffic forecasting. *NeurIPS*, 2020.

[13] C. Zheng, X. Fan, C. Wang, and J. Qi. GMAN: A graph multi-attention network for traffic prediction. *AAAI*, 2020.

[14] J. Choi, H. Choi, J. Hwang, and N. Park. Graph neural controlled differential equations for traffic forecasting. *AAAI*, 2022.

[15] Z. Li et al. STEP: Pre-training-enhanced spatial-temporal graph neural network for multivariate time series forecasting. *KDD*, 2022.

[16] J. Jiang, C. Han, W. X. Zhao, and J. Wang. PDFormer: Propagation delay-aware dynamic long-range transformer for traffic flow prediction. *AAAI*, 2023.

[17] H. Liu et al. Spatio-temporal adaptive embedding makes vanilla transformer SOTA for traffic forecasting. *CIKM*, 2023.

[18] Z. Chen et al. TITAN: A spatiotemporal feature learning framework for traffic incident duration prediction. *arXiv:2401.xxxxx*, 2024.

[19] J. Hu, L. Shen, and G. Sun. Squeeze-and-excitation networks. *CVPR*, 2018.

[20] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger. Densely connected convolutional networks. *CVPR*, 2017.

[21] I. Loshchilov and F. Hutter. SGDR: Stochastic gradient descent with warm restarts. *ICLR*, 2017.

[22] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. Graph attention networks. *ICLR*, 2018.

[23] D. Xu et al. Inductive representation learning on temporal graphs. *ICLR*, 2020.

[24] L. Cai, K. Janowicz, G. Mai, B. Yan, and R. Zhu. Traffic transformer: Capturing the continuity and periodicity of time series for traffic forecasting. *Transactions in GIS*, 2020.

[25] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, and C. Zhang. Connecting the dots: Multivariate time series forecasting with graph neural networks. *KDD*, 2020.

[26] C. Song, Y. Lin, S. Guo, and H. Wan. Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting. *AAAI*, 2020.

[27] Y. Chen and J. Segovia-Aguas. Bidirectional spatial–temporal adaptive transformer for urban traffic flow forecasting. *IEEE TITS*, 2022.

[28] Z. Fang et al. When spatio-temporal meet wavelets: Disentangled traffic forecasting via efficient spectral graph attention networks. *ICDE*, 2023.

[29] S. Ji et al. Spatio-temporal self-supervised learning for traffic flow prediction. *AAAI*, 2023.

[30] S. Lan, Y. Ma, W. Huang, W. Wang, H. Yang, and P. Li. DSTAGNN: Dynamic spatial-temporal aware graph neural network for traffic flow forecasting. *ICML*, 2022.